Ejercicio-06-DISEÑO-Estacionamiento

Cierre: sábado, 1 de agosto de 2026, 00:00

← Volver al índice de ejercicios

DISEÑO **Ejercicio 06** **Guiado paso a paso**

Estacionamiento

Ejercicio 06 · DISEÑO · Estacionamiento

Bloque UML Diseño + Patrones · Modelación y Diseño de Sistemas 2026

Campo	Valor
Modalidad	Grupos de 3-4 alumnos
Duración	**75 minutos** + 5 min de cierre
Pre-requisito	Tener resuelto el **Ejercicio 01 (ANÁLISIS Estacionamiento)**
Entregable	Plantilla de entrega completada

Punto de partida

Este ejercicio **continúa** el Ejercicio 01. Asumimos que el grupo ya tiene:

- ✅ Diagrama de Contexto
- ✅ Modelo de Objetos de Dominio (con 7-8 conceptos)
- ✅ SSD del UC "Salir del estacionamiento"
- ✅ Diagrama de Estados de `Ticket`
- ✅ Matriz de validación del Triángulo de Larman

Si no lo tienen, usar la solución de referencia del Ejercicio 01 como input.

Enunciado

> El equipo de análisis entregó los artefactos del Ejercicio 01. Ahora **tu rol cambia**: pasás de **analista** a **diseñador de software**.
>
> Tu tarea: transformar el análisis en una **solución diseñada** que pueda implementarse. Esto implica:
>
> - Convertir el **Modelo de Dominio** (conceptual) en un **Diagrama de Clases de Diseño** (con métodos, visibilidad, tipos técnicos, decisiones de implementación)
> - Expandir el **SSD** (caja negra) en un **Diagrama de Secuencia interno** (mostrando objetos colaborando dentro del sistema)
> - **Aplicar patrones** (SOLID + GRASP + GoF) donde aporten valor
> - **Justificar** cada decisión de diseño

Objetivos del ejercicio

Producir **3 entregables nuevos**:

1. **Diagrama de Clases de Diseño** (con métodos, visibilidad, tipos)
2. **Diagrama de Secuencia interno** del UC "Salir del estacionamiento"
3. **Documento de decisiones** (SOLID + GRASP + GoF aplicados)

Cronograma sugerido (75 min)

Tiempo	Paso
0 – 5 min	Lectura grupal del enunciado + repaso del análisis previo
5 – 20 min	**Paso 1**: Refinar conceptos del dominio en clases de diseño (atributos con tipos, métodos)
20 – 35 min	**Paso 2**: Diagrama de Clases de Diseño completo (visibilidad, dependencias)
35 – 55 min	**Paso 3**: DS interno del UC "Salir del estacionamiento" (Controller + Service + Entity)
55 – 70 min	**Paso 4**: Aplicar patrones · identificar SOLID, GRASP, GoF · justificar
70 – 75 min	Cierre · puesta en común

Reparto de roles

Rol	Responsabilidad
Coordinador	Tiempo + foco
Modelador	Dibuja diagramas
Patternista	Identifica patrones y los justifica
Relator	Llena plantilla de entrega

Paso 1 · De conceptos del Dominio a Clases de Diseño (15 min)

Objetivo: para cada concepto del dominio, decidir:

- **Visibilidad** de cada atributo (`+ public`, `- private`, `# protected`, `~ package`)
- **Tipo técnico** de cada atributo (`int`, `String`, `LocalDateTime`, `BigDecimal`, etc.)
- **Métodos** que la clase debe tener (no getters/setters triviales — métodos de negocio)
- **Constructor** y precondiciones

Ejemplo de transformación

Análisis (Modelo de Dominio):

```
Ticket
  codigo
  horaIngreso
  horaPago
  estado
```

Diseño (Clase de Diseño):

```
Ticket
- codigo: String
- horaIngreso: LocalDateTime
- horaPago: LocalDateTime [0..1]
- horaEgreso: LocalDateTime [0..1]
- estado: EstadoTicket
- pago: Pago [0..1]
+ Ticket(codigo: String, plaza: Plaza)
+ marcarPagado(pago: Pago): void
+ marcarSalido(): void
+ calcularDuracion(): Duration
+ estaVigente(): boolean
+ permiteEgresoEn(momento: LocalDateTime): boolean
```

Decisiones a tomar para cada clase

- ¿Qué métodos públicos expone?
- ¿Qué atributos son privados?
- ¿Qué validaciones van en el constructor?
- ¿Qué métodos son derivados (calculados) y no almacenan estado?

Listo cuando

- [ ] Al menos 6 clases con métodos, visibilidad y tipos
- [ ] Identificaron al menos 2 enumeraciones (`EstadoTicket`, `MedioPago`)
- [ ] Cada clase tiene al menos 1 método de negocio (no solo getters)

Paso 2 · Diagrama de Clases de Diseño completo (15 min)

Objetivo: dibujar el diagrama de clases con:

- Cajas con 3 compartimientos (nombre, atributos, métodos)
- **Dependencias** y **asociaciones** con multiplicidad
- **Interfaces** si las usan
- **Clases abstractas** marcadas en itálica

Estructura sugerida (a grandes rasgos)

```
┌────────────────────────────┐
│       <<service>>          │
│  GestorEstacionamiento     │
├────────────────────────────┤
│ - ticketRepo: TicketRepo   │
│ - tarifaSvc: TarifaSvc     │
│ - pasarela: Pasarela       │
├────────────────────────────┤
│ + procesarSalida(...)      │
│ + emitirTicket(...)        │
└───────────────┬────────────┘
                │ usa
                ▼
         ┌─────────────────┐
         │     Ticket      │
         ├─────────────────┤
         │ ... atributos   │
         ├─────────────────┤
         │ ... métodos     │
         └─────────────────┘

      ... + Pago, Tarifa, Pasarela, Repos
```

Decisiones de diseño

- **Interfaces**: `TicketRepository`, `PasarelaPago` (para inversión de dependencia)
- **Capas**: separar `entity` (dominio) de `service` (lógica) de `repository` (persistencia)
- **Enums**: `EstadoTicket`, `MedioPago`, `TipoPago`

Listo cuando

- [ ] Diagrama con al menos **8 clases / interfaces**
- [ ] Al menos **2 interfaces** explícitas
- [ ] Capas separadas (entity, service, repository)
- [ ] Métodos con firma completa

Paso 3 · DS interno del UC "Salir del estacionamiento" (20 min)

Objetivo: tomar el SSD de análisis (caja negra) y **expandirlo** mostrando qué objetos internos colaboran.

SSD original (caja negra · del Ejercicio 01)

```
Conductor → :Sistema → :Pasarela
  insertarTicket()
  monto: Dinero
  ...
```

DS interno (a producir)

```
Conductor   :CajaUI   :GestorEstac   :TicketRepo   :Ticket   :CalcTarifas   :Pasarela
   |           |           |              |            |           |            |
   | insertarTicket(cod)
   |──────────►|
   |           | procesarSalida(cod)
   |           |──────────►|
   |           |           | obtenerPorCodigo(cod)
   |           |           |───────────►|
   |           |           |  t: Ticket
   |           |           |◄-----------|
   |           |           | calcular(t)
   |           |           |─────────────────────────────────►|
   |           |           |  monto                           |
   |           |           |◄----------------------------------|
   |           |   monto   |
   |           |◄----------|
   |   monto   |
   |◄----------|
   |           |           |          ... pago ...            |
```

Lo que cambia respecto al SSD

Aspecto	SSD (caja negra)	DS interno
Participantes	Actor + Sistema	Múltiples objetos colaborando
Mensajes	Operaciones del negocio	Llamadas a métodos internos
Granularidad	Análisis	Diseño detallado
Detrás del telón	Oculto	Visible

Listo cuando

- [ ] Al menos **4 objetos internos** colaborando (UI, Service, Repository, Entity)
- [ ] Mensajes con **firma de método completa** (no del negocio sino del software)
- [ ] Retornos con tipos de software (no de dominio puros)
- [ ] El flujo cubre **mínimo el camino feliz**

Paso 4 · Aplicar patrones (15 min)

Objetivo: identificar y justificar al menos:

- 1 principio **SOLID** aplicado
- 1 patrón **GRASP**
- 1 patrón **GoF**

Pistas para este caso

SOLID

- **Single Responsibility**: `Ticket` no procesa pagos ni calcula tarifas — solo conoce su estado
- **Dependency Inversion**: `GestorEstac` depende de la **interfaz** `PasarelaPago`, no de la implementación concreta
- **Open/Closed**: el cálculo de tarifa puede extenderse (tarifa nocturna, eventos especiales) sin modificar el código existente

GRASP

- **Controller**: `GestorEstacionamiento` es el controller — coordina la operación de salida
- **Information Expert**: `Ticket` sabe calcular su duración (`calcularDuracion()`) porque tiene `horaIngreso`
- **Creator**: `Ticket` crea sus `Movimiento`s porque los contiene
- **Pure Fabrication**: `CalculadorTarifa` no existe en el dominio — es invento de diseño para encapsular lógica de cálculo

GoF

- **Strategy**: el cálculo de tarifa puede tener distintas estrategias (`TarifaPorHora`, `TarifaDiaria`, `TarifaCortesia`)
- **State**: `Ticket` cambia su comportamiento según estado (`Emitido`, `Pagado`, `Vencido`)
- **Factory Method**: `Pago` con subtipos (`PagoTarjeta`, `PagoEfectivo`) — factory los crea según medio

Listo cuando

- [ ] 1 SOLID identificado y justificado
- [ ] 1 GRASP identificado y justificado
- [ ] 1 GoF identificado, justificado y con esqueleto de diagrama

---

Paso 5 · Cierre (5 min)

Cada grupo, en 1 minuto:

1. ¿Qué clase del análisis tuvieron que **partir en dos** durante el diseño?
2. ¿Qué patrón GoF eligieron y por qué?
3. ¿Qué decisión de diseño les generó más debate?

Criterios de evaluación

Aspecto	Insuficiente	Aceptable	Bien
Clases de diseño	<6 clases sin métodos	6-8 con métodos básicos	8+ clases con métodos + interfaces
DS interno	<3 objetos	4-5 objetos colaborando	4-5 + retornos tipados + ciclo completo
SOLID	Mencionados pero no justificados	1 principio justificado	2+ principios con ejemplo concreto
GRASP/GoF	0 patrones	1 GRASP + 1 GoF	2 GRASP + 1-2 GoF + diagrama
Documentación	Sin justificación	Justificación breve	ADR-like con alternativas evaluadas

---

¿Qué hago si me atraso?

- **Paso 1**: 5 clases mínimo con métodos de negocio
- **Paso 2**: diagrama parcial con 6 clases
- **Paso 3**: DS con 3 objetos (UI + Service + Entity)
- **Paso 4**: 1 SOLID + 1 GRASP justificados (omitir GoF si no llegan)

Recursos

- Material C17 (Clases + SOLID), C18 (GRASP), C19 (GoF)
- Solución de referencia del Ejercicio 01 si necesitan el análisis como input
- Libro: Larman cap. 16-18; Martin (SOLID); GoF cap. 3-5

Modelación y Diseño de Sistemas 2026 · ESCMB-AUSI · UNC
Ing. Fernando Bono

01-Enunciado-y-Guia.pdf 26 de junio de 2026, 13:10

Agregar entrega

Estado de la entrega

Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar
Tiempo restante	La Tarea está retrasada por: 84 días 20 horas